SIMATS ENGINEERING

ASSIGNMENT 1 REPORT

Campus Smart-Parking System

Course: Java Programming

Course Code: CSA0909

Student Name: NALIPI REDDY ROHITH

Register Number: 192425115

GitHub Repository Link: https://github.com/nalipireddyrohith-web/CSA0909-java-Programming

Declaration and Submission Note

This report presents the analysis and proposed object-oriented design for the Campus Smart-Parking System in Tasks 1–12. Replace the student details and GitHub URL on the cover page. The test results shown are expected results derived from the stated test cases and supplied assignment examples; execute the Java programs on your machine and add genuine terminal screenshots before submission. Do not claim screenshots or GitHub commits that have not been created.

Abstract

The Campus Smart-Parking System manages parking floors and slots, vehicle entry and exit, ticket creation, parking charges, charging fees for electric vehicles, and parking passes. This report analyses the problem, selects data representations, gives pseudocode, proposes UML class structures, applies inheritance, overriding, polymorphism, abstraction and encapsulation, and evaluates the design against the changed requirements in Task 12. The recommended design uses a jagged array for floors with unequal slot counts, a common Vehicle abstraction, capability interfaces, and separate pricing rules to limit changes when new requirements are introduced.

Contents

- Task 1 – Problem Analysis and Candidate Objects
- Task 2 – Data Representation
- Task 3 – Control Logic and Pseudocode
- Task 4 – Classes, Objects and Constructors
- Task 5 – Static Members and Object Life Cycle
- Task 6 – Inheritance Hierarchy and UML
- Task 7 – Overriding and Object Methods
- Task 8 – Polymorphism
- Task 9 – Abstraction and Interfaces
- Task 10 – Packages, Access Modifiers and Encapsulation
- Task 11 – Implementation and Testing
- Task 12 – Re-evaluation and Design Challenge
- Overall Complexity Summary
- Conclusion
- Appendix – Execution and GitHub Checklist

Task 1 – Understand the Problem and Identify the Objects

1.1 Inputs and outputs

Inputs	Outputs
Registration number, vehicle category, user category/pass validity, entry and exit time, floor/slot configuration, charging energy in kWh	Allocated floor and slot, unique ticket number, entry record, parking fee, charging fee, released slot, daily collection total
Operation requested: vehicle entry or exit	Clear error when no slot is available, a registration is duplicated, or a floor/slot is invalid

1.2 Candidate classes and responsibilities

Candidate class	Responsibility and justification
ParkingFacility	Owns the floor/slot layout and allocates/releases slots. Required because facility state and slot operations should be managed in one place.
ParkingSlot	Represents one numbered slot, its floor, suitability and occupancy. Prevents two vehicles from using the same slot.
Vehicle (abstract)	Stores common registration data and defines common fee behaviour. Abstract because a generic vehicle has no single concrete rate.
TwoWheeler / Car / ElectricCar	Represent vehicle-specific rates or capabilities. Separate classes allow specialised behaviour without repeatedly checking a type field.
ParkingTicket	Stores a unique ticket ID, vehicle, slot, entry time and exit/billing details. Required to connect an entry with its later exit.
ParkingService	Coordinates entry, exit, duplicate-registration checks and ticket lifecycle.
PricingRule	Encapsulates fee policy so a new time-based rule can be introduced without changing every vehicle class.
ParkingPass / Pass interface	Represents whether a user has a valid monthly or one-day pass and the benefit it provides.
Chargeable interface	Represents the optional EV charging capability, independent of the vehicle's base parking fee.

1.3 Nouns, verbs and business rules

Candidate nouns include facility, floor, slot, vehicle, registration number, ticket, user, pass, fee and charging session. Candidate verbs become methods such as allocate(), release(), issueTicket(), calculateFee(), chargingFee(), enter(), exit() and validatePass().

- A slot may contain at most one vehicle; a free slot is represented by null or an explicit FREE state.
- A registration number must be unique among currently parked vehicles.
- Ticket identifiers must be unique and must not be reused during the application's lifetime.
- Exit must refer to an active ticket and a valid occupied slot; a released slot cannot be released twice.
- Parking duration cannot be negative. The billing rule in the supplied FeeDemo rounds every started hour up to a full hour.
- Only a valid pass receives the configured discount/free-parking benefit. EV charging is charged separately from parking.
- If no compatible slot is free, allocation fails without changing occupancy.

1.4 Assumptions

Registration numbers are normalised to uppercase and trimmed before comparison. Time is stored using java.time.Instant or LocalDateTime, with one consistent campus time zone. The simple fee example charges each started hour in full. Monthly-pass eligibility is checked by a pass service. Slot compatibility rules can be added for EV charging bays and vehicle size. Currency is Indian rupees (INR).

1.5 Alternative: objects versus parallel arrays

With objects, a Vehicle keeps registration, category, and behaviour together. Parallel arrays such as regNo[], type[], and entryTime[] require every index to remain aligned; adding an attribute requires another array and careful updates. A linear search by registration is O(n) in both designs without an index. An object model uses O(1) references plus each object's fields; parallel arrays use O(n) storage across their arrays but often have less per-record object overhead. A HashMap<String, ParkingTicket> improves average registration lookup to O(1), at the cost of additional index memory.

Task 2 – Choose the Data Representation

Data	Recommended type	Reason
Floor and slot index	int	Array indexes and counts are integral values.
Registration number	String	Alphanumeric plate; do not perform arithmetic on it.
Ticket number	long	Supports a large monotonically increasing identifier.
Entry/exit time	Instant or LocalDateTime	Represents date and time without using floating-point values.
Duration	long minutes or Duration	Avoids fractional-minute rounding.
Money	BigDecimal (production) or double (assignment demo)	BigDecimal avoids binary floating-point currency rounding; double is acceptable for the supplied output example.
Charging energy	double or BigDecimal	Represents kWh; validate that the value is non-negative.
Occupancy	boolean or nullable Vehicle	An explicit state is clear; null in a String[][] means free in the supplied skeleton.

2.1 Array representation

Use String[][] or ParkingSlot[][] as a jagged array: each floor has its own array length. For example, new int[]{20, 30, 50} represents three floors with different capacities. A rectangular array is less suitable if floors do not have equal slot counts.

2.2 Casting and billing precision

Integer division truncates: 61 / 60 equals 1, which undercharges a 61-minute stay when each started hour is billed. Use ceiling division with integers: billableHours = (minutes + 59) / 60 for minutes >= 0. For 130 minutes, (130 + 59) / 60 = 3 hours. Avoid casting minutes / 60 to double after the integer division, because the lost fractional part cannot be recovered.

2.3 Alternative data structures and complexity

Representation	Slot lookup / allocation	Space	Trade-off
Jagged array slots[floor][slot]	Scanning is O(N), where N is total slots	O(N)	Natural for unequal floor sizes; simple and easy to validate.
Flat one-dimensional array	O(N) scan; index conversion is O(1) when each floor has equal capacity	O(N)	Formula floor × slotsPerFloor + slot fails directly when floor capacities differ unless prefix offsets are maintained.
ArrayList<ParkingSlot[]>	Scan O(N); floor lookup O(1)	O(N)	Convenient when floor arrays are managed dynamically; still requires scanning for a free slot.

Task 3 – Design the Control Logic

3.1 Pseudocode: allocate a free slot

```
ALGORITHM Allocate(vehicleNo)
IF vehicleNo is null or blank THEN RETURN INVALID_INPUT
FOR each floor f from 0 to numberOfFloors - 1
    FOR each slot s from 0 to slots[f].length - 1
        IF slots[f][s] is null THEN
            slots[f][s] ← vehicleNo
            RETURN (f, s)
        END IF
    END FOR
END FOR
RETURN NO_FREE_SLOT
END ALGORITHM
```

3.2 Pseudocode: release a slot

```
ALGORITHM Release(floor, slot)
IF floor < 0 OR floor >= numberOfFloors THEN RETURN false
IF slot < 0 OR slot >= slots[floor].length THEN RETURN false
IF slots[floor][slot] is null THEN RETURN false
slots[floor][slot] ← null
RETURN true
END ALGORITHM
```

3.3 Pseudocode: calculate parking fee

```
ALGORITHM CalculateFee(minutes, hourlyRate, dailyMaximum)
IF minutes < 0 THEN RETURN INVALID_DURATION
IF minutes = 0 THEN RETURN 0
hours ← (minutes + 59) DIV 60
fee ← hours × hourlyRate
IF dailyMaximum is configured THEN fee ← MIN(fee, dailyMaximum)
RETURN fee
END ALGORITHM
```

A pass discount is applied only after validating the pass. If the monthly pass makes parking free, parking fee becomes zero; charging fees remain separate unless the policy explicitly waives them. A return exits the operation as soon as the answer is known. A break can stop a search loop after finding a free slot; continue can skip incompatible slots.

3.4 Completed supplied ParkingFacility logic

```
public int[] allocate(String vehicleNo) {
    for (int f = 0; f < slots.length; f++) {
        for (int s = 0; s < slots[f].length; s++) {
            if (slots[f][s] == null) {
                slots[f][s] = vehicleNo;
                return new int[] { f, s };
            }
        }
    }
    return null;
}
public boolean release(int floor, int slot) {
    if (floor < 0 || floor >= slots.length) return false;
    if (slot < 0 || slot >= slots[floor].length) return false;
    if (slots[floor][slot] == null) return false;
    slots[floor][slot] = null;
```

```
        return true;
    }
```

Expected output for the assignment's provided main(): [0, 0], [0, 1], [1, 0], null, true false false, [0, 1].

3.5 Alternative: free-slot stack

Maintain a stack of free (floor, slot) pairs. Allocation pops one pair and marks it occupied; release validates the slot, marks it free and pushes its pair. Both operations are O(1) average time with a stack, compared with O(N) allocation scanning and O(1) release validation in an array. The stack adds O(N) storage for free-slot references/pairs; the slot map itself still uses O(N) space. A free-count per floor helps identify full floors, but counts alone do not identify the actual free slot.

Task 4 – Design Classes, Objects and Constructors

Vehicle is initialised with a valid registration number. ParkingTicket receives its vehicle, slot, ticket ID and entry time through a parameterised constructor. Constructor chaining with this(...) avoids duplicated validation logic.

```
class ParkingTicket {
    private final long ticketId;
    private final Vehicle vehicle;
    private final int floor;
    private final int slot;
    private final java.time.Instant entryTime;
    public ParkingTicket(long id, Vehicle v, int f, int s, java.time.Instant entry) {
        if (id <= 0 || v == null || f < 0 || s < 0 || entry == null)
            throw new IllegalArgumentException("Invalid ticket data");
        this.ticketId = id; this.vehicle = v; this.floor = f;
        this.slot = s; this.entryTime = entry;
    }
}
```

A default constructor is appropriate only when a meaningful valid default state exists. Setters after a default constructor can expose a partially initialised object; a parameterised constructor is safer for immutable ticket data. A static factory method such as ParkingTicket.issue(...) can validate policy or choose a subclass before construction. A copy constructor is usually unnecessary for tickets because duplicating a ticket ID would violate uniqueness; a printed/reissued copy should retain a reference to the same logical ticket or receive a new reissue identifier.

Local reference variables and method frames are generally on the stack, while objects are allocated on the heap. Exact JVM allocation and optimisation are implementation-dependent. Creating n tickets takes O(n) total time and O(n) storage for the n ticket objects, excluding referenced vehicle and index structures.

Alternative comparison

Creation approach	Validity	Time per object	Main risk
Parameterised constructor	Can enforce all required invariants before return	O(1) for fixed fields	Constructor may become long if too many unrelated concerns are combined.
Default constructor + setters	Can be temporarily invalid	O(1) plus each validation	Partially initialised state; callers may forget setters.
Static factory	Can validate and return subtype or cached result	O(1) for fixed work	Factory logic must remain clear; does not remove the need for valid constructors.

Task 5 – Apply Static Members and the Object Life Cycle

A static ticket counter is shared by all instances of the class that declares it. For example, private static long nextTicketId = 1; and a synchronized or atomic incrementing method can issue unique IDs in a concurrent application. If tickets should be globally unique across multiple application servers, a database sequence or UUID is safer than an in-memory counter.

Static methods suit operations that do not require one particular object, such as validating a registration format or issuing a globally scoped ID. A static block may initialise immutable configuration, but should not be used for complicated application startup.

After a vehicle exits, the active-ticket map and facility slot must release references to the ticket/vehicle when no longer needed. The object becomes eligible for garbage collection only when no reachable reference keeps it alive; garbage collection timing is not guaranteed. Methods pass object references by value and may return an object reference.

Facility counter alternative

If the counter is an instance field of ParkingFacility, k facilities each keep a separate counter, requiring O(k) counter storage and allowing duplicate ticket IDs between facilities. If IDs need only be unique within each facility, a per-facility counter is valid when ticket IDs include the facility ID. If IDs must be globally unique in one JVM, a static counter uses O(1) counter storage for k facilities and n tickets; the ticket objects themselves still require O(n) storage.

Task 6 – Design the Inheritance Hierarchy

Use hierarchical inheritance: TwoWheeler and Car extend Vehicle; ElectricCar extends Car and implements Chargeable. Vehicle holds shared registration state. Subclasses provide a rate or specialised policy. User categories can be modelled separately with a ParkingPass capability rather than forcing users into the vehicle hierarchy.

6.1 UML class diagram (text UML)



UML relationships: TwoWheeler inherits Vehicle; Car inherits Vehicle; ElectricCar inherits Car and realises Chargeable. ParkingFacility aggregates ParkingSlot objects; ParkingTicket refers to one Vehicle and one allocated slot.

6.2 Use of super and constructor trace

```
class Car extends Vehicle {
    Car(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 30; }
}
class ElectricCar extends Car implements Chargeable {
    ElectricCar(String regNo) { super(regNo); }
    public double chargingFee(int kWh) { return 12.0 * kWh; }
}
```

Constructor trace for new ElectricCar("TN03C3"): Object constructor → Vehicle constructor (via Car's super chain) → Car constructor → ElectricCar constructor. The Vehicle constructor initialises regNo before subclass construction completes. super(...) must be the first constructor statement when explicitly used.

6.3 Inheritance versus composition

With inheritance, a new vehicle type normally requires a new subclass and its overridden rate method. With composition, Vehicle has-a PricingRule or VehicleType object. Composition is more flexible when rates change independently of vehicle identity and reduces subclass growth. Fee calculation remains O(1) for either design when the selected rate/rule is already available. Inheritance has one object plus inherited state; composition adds a reference and usually a policy object, increasing per-vehicle reference storage by O(1).

Task 7 – Override and Specialise Behaviour

Subclasses override ratePerHour() so the same calculateFee() operation uses the concrete vehicle's rate. The @Override annotation lets the compiler catch signature mistakes. An overriding method must have a compatible signature and covariant/compatible return type, and cannot reduce access visibility.

Use final for immutable fields such as regNo and ticketId. A method may be final if policy must not be overridden; a class may be final when extension is intentionally prohibited. Avoid final on Vehicle if the design must support future vehicle types.

equals() and hashCode() must use the same identity rule. If registration number is the chosen identity, normalise it first. The supplied FeeDemo compares Car("TN02B2") with ElectricCar("TN02B2") and prints true because both inherit Vehicle.equals() and share the same registration string. This can be correct if a registration identifies one vehicle regardless of Java subtype, but it can hide inconsistent type data; the production system should reject a duplicate active registration before creating another active record.

Switch alternative and complexity

A single calculateFee() method with a switch on vehicle type has O(1) time for a fixed switch with a small finite set of types, just as virtual dispatch is effectively O(1) for this model. The main difference is extensibility: adding a type can require modifying the central switch and all related switches. With overriding, the new subclass supplies its behaviour and existing code that depends on Vehicle can remain unchanged. This is a maintainability and change-locality benefit rather than an asymptotic time improvement.

Task 8 – Apply Polymorphism

Compile-time polymorphism is method overloading, for example calculateFee(int minutes) and calculateFee(LocalTime start, LocalTime end). The compiler chooses the overload using argument types at compile time. Runtime polymorphism occurs when a Vehicle reference points to a TwoWheeler, Car or ElectricCar and the overridden ratePerHour() method is selected at runtime.

```
public double calculateFee(int minutes) {
    if (minutes < 0) throw new IllegalArgumentException("Negative minutes");
    int hours = (minutes + 59) / 60;
    return hours * ratePerHour();
}
```

Dispatch trace: declared reference type = Vehicle; actual object type = ElectricCar; call = v.calculateFee(130); inherited Vehicle.calculateFee() calls ratePerHour(); dynamic dispatch executes Car.ratePerHour() (30) for ElectricCar because ElectricCar inherits the car rate; 3 billable hours × Rs 30 = Rs 90 parking fee. Then Chargeable.chargingFee(10) adds Rs 120, producing Rs 210 total.

Completed FeeDemo calculations

Vehicle	Minutes	Billable hours	Parking fee	Charging fee	Total
TwoWheeler TN01A1	130	3	3 × Rs 10 = Rs 30	Rs 0	Rs 30.0
Car TN02B2	130	3	3 × Rs 30 = Rs 90	Rs 0	Rs 90.0
ElectricCar TN03C3	130	3	3 × Rs 30 = Rs 90	10 × Rs 12 = Rs 120	Rs 210.0

Expected output:

```
TwoWheeler TN01A1 -> Rs 30.0
Car TN02B2 -> Rs 90.0
ElectricCar TN03C3 -> Rs 210.0
true
```

Using instanceof checks and casts instead would require a branch for each category. A linear chain can perform up to k checks for k categories, O(k) in the number of categories in the worst case, and central code must be edited for every new category. Dynamic dispatch is effectively constant-time for this design and keeps the category-specific behaviour local.

Task 9 – Introduce Abstraction

Vehicle should be abstract because it represents common state and a contract for all vehicle types, while ratePerHour() is implemented by concrete types. Chargeable should be an interface because charging is an optional capability: some vehicles can be chargeable while others cannot. A class can implement multiple interfaces, such as ElectricCar implementing Chargeable and a future Insurable capability.

An interface default method is useful for a shared optional implementation; a static interface method can provide a related utility. Neither replaces the need for an abstract class when shared instance state and constructors are required.

If Vehicle became an interface

An interface can declare abstract/default/static methods but cannot have ordinary per-object instance fields or constructors. Vehicle state would have to move into implementing classes or composition. Java classes can implement multiple interfaces but can extend only one class; turning Chargeable into an abstract class could prevent ElectricCar from extending both Car and that class. Per-call runtime cost is generally not meaningfully different for this small model; each object would still need storage for its state, and composition may add references.

Task 10 – Packages, Access Modifiers and Encapsulation

10.1 Suggested package structure

```
parking.model: Vehicle, TwoWheeler, Car, ElectricCar, ParkingTicket, ParkingSlot, ParkingPass
parking.pricing: PricingRule, HourlyPricingRule, DeliveryVanPricingRule
parking.service: ParkingFacility, ParkingService, TicketService
parking.app: Main / command-line user interface
```

10.2 Access modifiers

Modifier	Use in this design	Reason
private	Fields such as regNo, ticketId, slot occupancy	Protects state from direct external modification.
protected	ratePerHour() hook for subclasses	Lets subclasses implement specialisation without exposing it as public API.
public	Constructors and service methods intended for other packages	Defines the supported external interface.
package-private (default)	Helper classes or methods used only inside one package	Limits accidental use outside the package.

Getters expose read-only values where appropriate. Setters validate before changing state; immutable values such as ticketId should not have setters. Reject negative duration, null registration numbers, negative kWh and invalid floor/slot indexes. Use Integer.equals() or unbox safely rather than comparing Integer objects with ==, which can compare object identity rather than numeric value.

10.3 Public fields alternative

Public fields allow callers to set invalid values and bypass validation. Validation would need to be repeated at every call site, making omissions likely. A simple validation check costs O(1) per update, but the repeated maintenance burden grows with the number of callers. Encapsulation centralises the rule in one place.

Task 11 – Implement, Test and Defend the Design

11.1 Test plan

Run the tests against the implementation and record the actual output. The 'Actual result (expected before execution)' column below lists the predicted result from the stated design; replace it with the real observed result and attach screenshots after executing your code.

Test ID	Scenario / input	Expected result	Actual result to record
T11-01	TwoWheeler parks for 130 minutes at Rs 10/hour	Rs 30 parking fee	Pending local execution
T11-02	Car parks for 130 minutes at Rs 30/hour	Rs 90 parking fee	Pending local execution
T11-03	ElectricCar parks for 130 minutes and uses 10 kWh	Rs 90 parking + Rs 120 charging = Rs 210	Pending local execution
T11-04	Facility has 3 slots; allocate 4 vehicles	First 3 allocated; fourth returns null	Pending local execution
T11-05	Release valid occupied slot twice	true on first release, false on second	Pending local execution
T11-06	Release with floor = 5 when only 2 floors exist	false; no state change	Pending local execution
T11-07	Parking duration = 0 minutes	Rs 0 under the stated formula	Pending local execution
T11-08	Parking duration = 60 minutes	Exactly 1 billable hour	Pending local execution
T11-09	Parking duration = 61 minutes	2 billable hours	Pending local execution
T11-10	Duplicate active registration enters again	Rejected; no second active ticket	Pending local execution
T11-11	Unknown vehicle category	Rejected with validation message	Pending local execution
T11-12	Valid monthly pass for parking	Parking fee waived; charging policy handled separately	Pending local execution

11.2 Hand-calculated daily collection example

Assume three completed visits: one TwoWheeler parks 130 minutes (3 × Rs 10 = Rs 30), one Car parks 130 minutes (3 × Rs 30 = Rs 90), and one ElectricCar parks 130 minutes (3 × Rs 30 = Rs 90) and consumes 10 kWh (10 × Rs 12 = Rs 120). Total collection = Rs 30 + Rs 90 + Rs 90 + Rs 120 = Rs 330. This example assumes no pass discount and no daily maximum. Confirm the same total from the program output.

11.3 Design evaluation

The design uses encapsulation by keeping fields private, inheritance to reuse common vehicle state, overriding for vehicle rates, polymorphism through Vehicle references, and abstraction through Vehicle and Chargeable. A limitation of a simple array-based implementation is that finding an active ticket by registration is O(n), and concurrency is not handled unless operations are synchronised or protected by a suitable locking strategy.

11.4 Active ticket storage alternatives

Structure	Exit lookup	Space for n active vehicles	Notes
Array / ArrayList<ParkingTicket>	O(n) scan by registration	O(n)	Simple and compact; ArrayList append is amortised O(1), removal by index can shift O(n) elements.
HashMap<String, ParkingTicket>	Average O(1), worst-case can degrade	O(n) plus hash table overhead	Recommended for active tickets keyed by normalised registration; remove on exit.
Array plus registration index map	Average O(1) lookup through map	O(n) records + O(n) index	Useful if ordering/reporting and fast lookup are both needed.

Task 12 – Re-evaluate and Adapt the Design

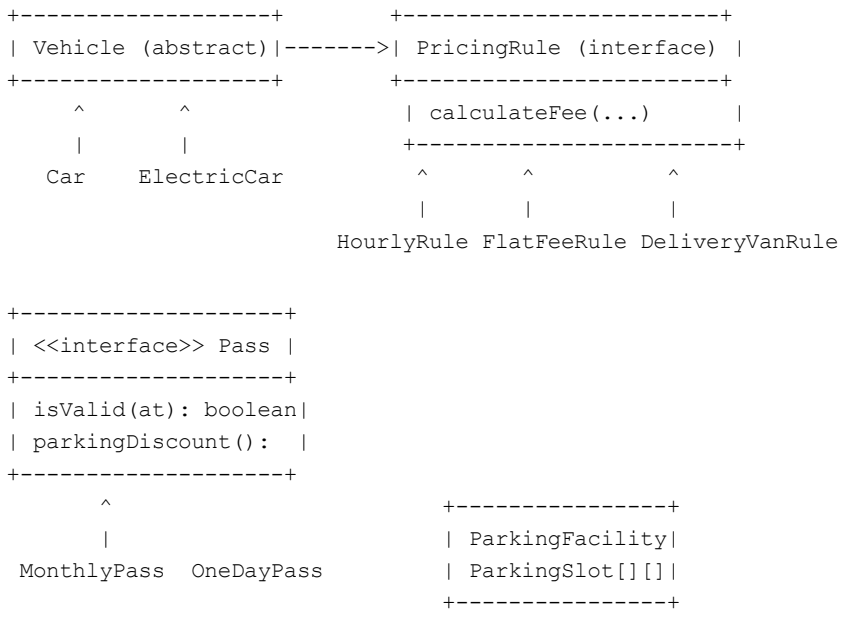
12A. Impact analysis

- New e-bikes have a flat parking fee, so a simple hourly-rate-only design is insufficient unless it supports an alternative pricing strategy.
- Delivery vans cost double the car rate from 9 AM to 5 PM, requiring time-dependent pricing based on the parked period and applicable local time.
- Visitors can purchase a one-day pass, so pass validity must represent a date/time window and user category rather than only a monthly-pass boolean.
- The facility now has five floors with different slot counts. A jagged array is needed so each floor can have its own capacity.
- Classes likely affected in a tightly coupled design: Vehicle subclasses, fee calculation, pass validation, and facility initialisation. With a PricingRule abstraction and Pass interface, most existing tested classes can remain unchanged.

12B. Evaluate the original design

A subclass-per-vehicle design works for simple stable rates, but delivery-van pricing varies by time and e-bike pricing is flat rather than hourly. Embedding all rules in each subclass or a central switch spreads policy and makes changes risky. A design with pricing strategies and pass abstractions is more appropriate. Existing Vehicle, Car and TwoWheeler classes can remain unchanged if the new rules are introduced as separate strategies and new types.

12C. Revised class structure



Revised design: Vehicle delegates fee computation to a PricingRule; e-bike receives FlatFeeRule; cars use HourlyRule; delivery vans use DeliveryVanRule, which applies twice the car rate for the portion of parking in the 9 AM–5 PM window. Pass is a capability/contract, with MonthlyPass and OneDayPass implementations. ParkingFacility uses a jagged array sized from each floor's capacity.

12C.1 Illustrative Java code for new pricing and passes

```
import java.time.*;
interface PricingRule {
    double calculateFee(LocalDateTime entry, LocalDateTime exit);
}
final class FlatFeeRule implements PricingRule {
```

```
    private final double amount;
    FlatFeeRule(double amount) { if (amount < 0) throw new IllegalArgumentException();
this.amount = amount; }
    public double calculateFee(LocalDateTime entry, LocalDateTime exit) { return amount; }
}
final class HourlyRule implements PricingRule {
    private final double rate;
    HourlyRule(double rate) { if (rate < 0) throw new IllegalArgumentException(); this.rate =
rate; }
    public double calculateFee(LocalDateTime entry, LocalDateTime exit) {
        long minutes = Duration.between(entry, exit).toMinutes();
        if (minutes < 0) throw new IllegalArgumentException("Exit before entry");
        return ((minutes + 59) / 60) * rate;
    }
}
interface Pass { boolean isValid(LocalDateTime at); }
final class OneDayPass implements Pass {
    private final LocalDate date;
    OneDayPass(LocalDate date) { this.date = date; }
    public boolean isValid(LocalDateTime at) { return date.equals(at.toLocalDate()); }
}
```

DeliveryVanRule should split the interval into segments inside and outside 09:00–17:00 and apply 2 × car rate only to the overlapping minutes. The exact boundary rule (whether 09:00 and 17:00 are included) should be specified by the university. The code above is an illustrative foundation, not a complete production billing engine.

12D. Evidence-based defence

Evaluation criterion	Design defence
Existing classes changed	With policy composition, expected changes are limited to wiring/configuration and the Vehicle-to-PricingRule reference; Car/TwoWheeler fee logic need not change. Record the exact count from your implementation.
Adding a vehicle type	Add a new vehicle class or category and select a PricingRule; avoid modifying a large central switch.
E-bike test	For any valid entry/exit interval, FlatFeeRule returns its configured flat amount.
Delivery van test	A stay wholly within 09:00–17:00 costs twice the configured car rate for the billed interval; a crossing interval must split the calculation at the time boundaries.
One-day pass test	Pass is valid when the check time's local date matches the pass date; test before, on and after that date.
Trade-off	Strategy objects add a small amount of indirection and configuration, but reduce coupling and make pricing rules independently testable.

Overall Complexity Summary

Operation / structure	Time complexity	Auxiliary or stored space
Linear search for registration in n active tickets	O(n)	O(1) extra
HashMap lookup by registration (average)	O(1) average	O(n) map storage
Scan jagged array to allocate among N total slots	O(N) worst case	O(1) extra beyond slot storage
Release a known floor/slot in jagged array	O(1)	O(1)
Stack of free slots: allocate/release	O(1) average	O(N) stack storage
Fee calculation for one ticket	O(1) for fixed arithmetic/rule; delivery time segmentation depends on implementation	O(1)
Construct n ticket objects	O(n) total	O(n) ticket storage
Compare k vehicle categories with instanceof chain	O(k) worst case	O(1) extra

Conclusion

The initial object-oriented model satisfies the core parking workflow and demonstrates Java fundamentals. For the changed requirements, the recommended approach is to retain the useful domain objects but modify the design to use a jagged floor/slot representation, a pricing strategy abstraction, and a pass interface. This makes flat fees, time-window pricing and new pass types easier to add while limiting changes to existing tested classes. The final submission should include executable Java source for every task, real output screenshots, and a working GitHub repository.

Appendix – Execution and GitHub Checklist

- Create repository: CSA09-<RegisterNumber>-Assignment1.
- Create Task01 through Task12 folders, each with source files and a README or short result note.
- Compile and run each Java program with the Java version required by the course.
- Save genuine terminal screenshots showing input and output for every required task.
- Complete the supplied ParkingFacility and FeeDemo programs and compare outputs with the assignment's expected outputs.
- Record actual results in Task 11's test table; do not leave 'pending' in the submitted copy.
- Add the UML diagram (this report includes a text UML representation; a drawn UML image may be added if required by faculty).
- Commit progressively, with at least one meaningful commit per task.
- Paste the working GitHub repository URL on the cover page and in the submission field.